

PROJECT REPORT

STUDENT PERFORMANCE MANAGEMENT SYSTEM

A Python-Based Academic Record and Performance Analysis Application

Course	CSE1021 – Introduction to Computer Problem Solving and Programming
Student Name	Ritika Jain
Registration Number	_____
Faculty Name	_____
Academic Year	2026

Submitted as part of the evaluated course project.

1. Introduction

The Student Performance Management System is a command-line application developed in Python. It helps a user maintain student records and review basic academic performance. The program provides options to add, view, search, update and delete student records, calculate grades, and generate a summary report.

2. Problem Statement

Maintaining student details and calculating performance manually can take time and may lead to calculation or record-keeping errors. A small digital system can make these tasks more organised by storing student records in a file and performing repeated calculations automatically.

3. Objectives

- Maintain student names, roll numbers, marks and grades.
- Provide basic create, read, update and delete operations.
- Validate marks so that they remain within 0–100.
- Calculate grades and pass/fail status using defined thresholds.
- Generate a report showing the number of students, average, highest and lowest marks, and pass/fail counts.

4. Scope and Target Users

The project is intended for a small, local record set and is suitable as a learning application for teachers, students, or administrators who need basic academic record management. It runs in a terminal and stores data in a local text file. It does not currently provide a graphical interface, database server, or user-login system.

5. Functional Requirements

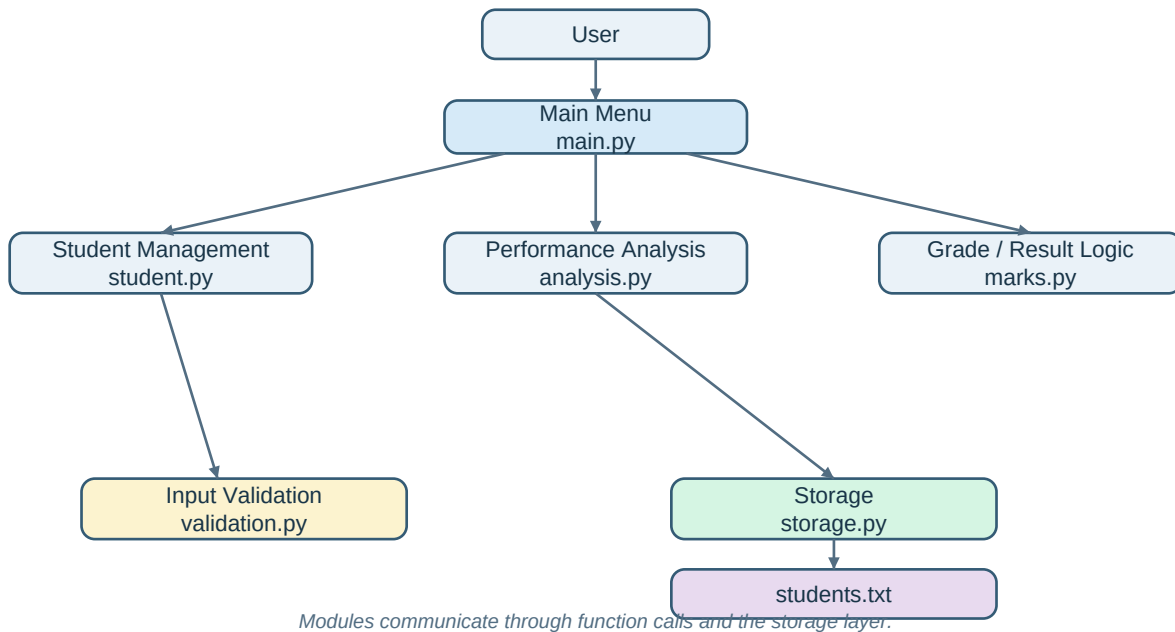
ID	Requirement	Description
FR-1	Add student	Accept a roll number, name and marks; calculate the grade and save the record.
FR-2	View records	Display the saved student records.
FR-3	Search student	Find a record using the roll number.
FR-4	Update student	Update a matching student's name and marks, then recalculate the grade.
FR-5	Delete student	Remove a matching student record.
FR-6	Performance report	Calculate average, highest/lowest marks, and pass/fail counts.

6. Non-Functional Requirements

- **Usability:** a numbered menu makes the functions easy to select.
- **Reliability:** records are saved to a local file so they remain available between runs.
- **Input validation:** marks are checked for numeric input and the 0–100 range; empty text inputs are rejected.
- **Maintainability:** related tasks are separated into Python modules.
- **Resource efficiency:** the application uses a small local file and standard Python libraries.

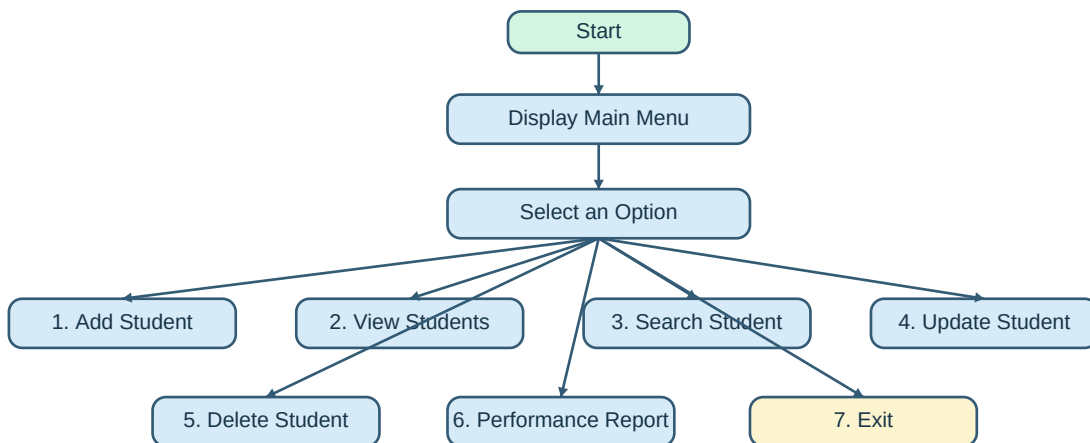
7. System Architecture

The application uses a modular structure. The main menu calls the student-management and reporting functions. Validation, grade calculation and storage are handled by separate modules.



8. Workflow Diagram

The user selects an operation from the menu. The selected operation runs and the program returns to the menu. Choosing Exit ends the program.



After an operation, the program returns to the menu; Exit ends the program.

9. Module Description

File	Responsibility
main.py	Displays the menu, accepts the selected option and calls the required function.
student.py	Implements adding, viewing, searching, updating and deleting student records.
marks.py	Calculates grade, percentage and pass/fail result.
validation.py	Validates marks, roll numbers and non-empty text input.
storage.py	Loads and saves records in students.txt using the CSV module.
analysis.py	Generates the overall performance report.
students.txt	Local data file containing one student record per line.

10. Implementation Details

The project is implemented using Python functions and modules. Conditional statements determine grades and menu actions. Loops repeatedly display the menu and process student records. File handling is used to persist data, and the CSV module reads and writes comma-separated fields. The program uses a roll number to locate a student's record.

Grade thresholds used by the program

Marks	Grade
90–100	A+
80–89.99	A
70–79.99	B
60–69.99	C
50–59.99	D
Below 50	F

11. Data Storage Design

Each line in students.txt stores four fields in this order: student name, roll number, marks, and grade. The storage module reads the fields into dictionaries for processing and writes the records back when data changes. The application uses a local file rather than a database.

12. Testing Approach and Results

The application was manually tested through the command-line menu. The following test cases describe the expected behaviour and the checks performed during development. Before final submission, run these tests again using the final version of all files.

Test	Action	Expected result	Status
T-1	Add a student with valid details	Record is saved and success message appears	Reported working
T-2	View all records	Saved record details are displayed	Reported working
T-3	Search using an existing roll number	Matching student is displayed	Reported working
T-4	Update an existing record	Updated name/marks and grade are saved	Reported working
T-5	Delete a test record	Matching record is removed	Test as applicable

Test	Action	Expected result	Status
T-6	Generate performance report	Summary statistics are displayed	Reported working
T-7	Enter an invalid menu option	Error message appears; menu repeats	Reported working

13. Challenges and Learning Outcomes

During development, the main practical tasks were organising the code into separate files, connecting modules through imports, preserving records in a text file, and checking input before processing it.

Key concepts applied include variables, data types, conditional statements, loops, functions, lists and dictionaries, file handling, CSV processing, exception handling, and modular programming. The project demonstrates how these concepts can be combined to solve a practical record-management problem.

14. Limitations

- The interface is text-based and runs in a terminal.
- Data is stored locally in a text file and is not encrypted.
- The system does not include authentication or role-based access.
- The current marks model treats marks as a single score out of 100.

15. Future Enhancements

- Add subject-wise marks and semester-wise performance.
- Create a graphical user interface.
- Export reports to CSV or PDF.
- Use SQLite for structured data storage.
- Add login, permissions and record backups.

16. Conclusion

The Student Performance Management System demonstrates a basic but practical use of Python for managing academic records. It combines modular functions, input validation, file handling and simple statistical calculations in one command-line application. Further improvements could make it more suitable for larger datasets and everyday use.

17. References

1. Python Software Foundation, Python 3 documentation: <https://docs.python.org/3/>
2. CSE1021 – Introduction to Computer Problem Solving and Programming course concepts and project requirements provided for this assignment.

Before submission: Fill in your registration number and faculty name on the cover page. Add genuine screenshots of your running application if required by your course instructions. Ensure the report matches the final code in your GitHub repository.